

AIC·AI+开源技术报告

- 团队名称：（报名系统内填写）
- 团队编号：（报名系统内填写）
- 作品名称：Icarus 智测
- 参赛方向：（二）AI 开发工具与开源协作
- 日期：2026 年 月 日

作品速览

要解决的问题	写得出 RTL，写不动测试台；AI 能代写 testbench，但它同时会编期望值——期望值一旦是错的，就会把错的设计判成通过。让被考核的对象兼任考官，是这类做法最根本的缺陷。
我们怎么做	把"提出"与"裁决"分成两个可分别审计的部分：AI 只产出受严格 Schema 约束的测试计划，PASS/FAIL 只由开源仿真器 Icarus 的真实编译与仿真给出；AI 写的期望值被独立的确定性参考模型复算并覆盖。
做到了什么	15 案例 / 83 缺陷 83/83 检出、0 误报、0 不可判定；98 变体 98/98 可综合；离线无密钥跑通完整 AI 路径；598 项测试 0 失败。
最硬的一条证据	四档消融（同一批 83 缺陷）：关掉参考模型预言机，AI 计划检出 76 → 0——权威预言机不是修辞，是承重墙（\$5.3）。
开放成果	Apache-2.0；CLI + 库 + 网页 + 可复用 GitHub Action；基准集与 44 条带正反例的规则；《开源及第三方资源使用清单》。
如实说明	不做时序签核/形式化/覆盖率；参考模型只覆盖内置案例；对象③④需求未调研（\$1.0、\$7.4）。

- 说明：本文按《AIC·AI+开源技术报告参考大纲》九节组织，正文 19 页，超出大纲建议的
- 15 页。我们选择不为此删证据：多出的篇幅用于消融实验、三个真实事故复盘、测试条件表与
- 对比选型表，而大纲明确写了"不以报告篇幅、图片数量或材料数量作为评分依据"。
- 更细的过程材料（逐条根因、完整时间线、漏检明细）已移入附录。正文每一项数字都可在随附
- 仓库复现（\$9.9）。评审材料已按竞赛规则第三条第（九）款做匿名处理。

一、作品概述

1.0 作品服务谁（先说清楚，后面所有取舍都从这里来）

一个工具能服务谁，取决于它把什么能力暴露出来了。本项目按"要完成的任务"识别出四类对象，两类主打：

对象	要完成的任务	为什么我们合适
① 用 AI 生成 RTL 的开发者（主打）	"大模型给的 Verilog 能编译，但它到底对不对？"今天的做法是让同一个模型再写一遍 testbench——让被考核的对象兼任考官	第三门就是为"AI 的数字不能作数"设计的：参考模型复算覆盖 AI 期望值、三态证据等级、四层结论口径
② 开源 RTL 维护者 / 贡献者（主打）	"这个 PR 重写了模块，行为变了吗？可上游根本没有 testbench"	verify-diff 不需要被测项目自带 testbench：从 module 头自动提合约、生成计划，两份 RTL 跑同一份计划后逐项 + 逐拍比对
③ 数字电路 / Verilog 学习者	"测试台写不对，报错也看不懂"	完全离线、无需密钥、有中文手册与 15 个可对照案例；失败被翻成"下一步看哪一行"
④ 课程教师 / 助教	"50 份作业要批，判定要一致，学生不服要能解释"	确定性判定 + 逐条证据 + 批量批改与相似度线索 + 全程离线（不把学生代码传云端）

为什么主打 ①②：它们共用同一套底层能力（权威预言机 + 免 testbench 的行为等价对比），所以为一个对象做的适配几乎原样服务另一个；而它们又正好分别对应本赛道的两个关键词——"AI"与"开源"。对象 ③④ 用同一套能力的不同包装覆盖。

诚实标注（未达标项前置）：

对象 ③④ 的用户需求来自真实试用反馈与课程场景推断，尚未做正式调研，本报告不对其需求强度下结论；

对象 ④ 的批量批改已实现（iverilog-ai grade：批量行为对比 + 两两相似度线索表），但没有在真实班级里试用过，因此不声称它在 50 人规模下的实际体验；

正文 19 页，超出大纲建议的 15 页（见文首说明），我们没有为此删除证据；

上游 Issue / Pull Request 尚未提交（见 §6.3）；

其余未达标项集中列在 §7.4。

1.1 背景与问题

围绕 Icarus Verilog 做验证的人，几乎都会卡在同一处：写得出 RTL，写不动测试台。

我们在自己的数字电路学习与项目实践中反复遇到三个具体问题：

- (1) 测试台的门槛高于设计本身。一个 30 行的模十计数器，要覆盖复位、使能保持、回绕三类行为，手写测试台通常需要 80 行以上，而且每换一个设计就要重写一遍。初学者的常见结果是"RTL 写完了，但只做了肉眼检查"。
- (2) "AI 说验证过了"无法核验。通用大模型可以直接生成 testbench，但它同时会生成

期望值——而期望值一旦是錯的，錯的期望值會把錯誤的設計判成"通過"，或者把正確的設計判成"失敗"。這不是假設：本項目在真實在線模型實驗中就觀測到過這類現象（詳見 §5.3）。讓被考核的對象同時擔任考官，是這套做法最根本的缺陷。

這一條在 2026 年變得比以往更迫切：AI 已經能寫出一份能編譯、看起來合理的 RTL，唯一的缺口是"沒人驗"。而驗收一份 AI 改寫的 RTL，恰恰是最需要"比兩份實現"而不是"從頭寫一個測試台"的場景——這正是對象 ①② 共用的那套能力。

(3) 現有工具鏈缺一端自動化。Icarus Verilog 是成熟、可靠、完全開源且可腳本化的仿真器，它的定位是"給定 testbench，給出結果"。從"我想驗證什麼"到"可執行的 testbench"這一段，目前沒有開源工具覆蓋。上游開源 RTL 項目普遍沒有測試台，也就沒有任何低成本的行为回歸手段。

1.2 目標、參賽方向與作品形態

參賽方向：(二) AI 開發工具與開源協作。

作品形態：命令行工具 + 網頁應用 + GitHub Action + 可复用基準集與規則包。

核心功能：把自然語言的驗證目標，轉換為受嚴格約束的結構化測試計劃，再由確定性代碼生成 testbench，交給 Icarus Verilog 真實編譯與仿真，最後產出可追溯的證據與報告。針對對象 ②，另有一條不需要輸入測試台的路徑：從 RTL 自動提合約、生成計劃，把同一份計劃喂給兩份 RTL，判定行为是否一致。

一句話概括本項目的分工主張：AI 提出測試假設，開源仿真器作出判決。

1.3 核心亮點

AI 不參與判分。PASS/FAIL 只由 Icarus 的編譯/仿真結果與結構化斷言決定；AI 的期望值被獨立的不確定性參考模型復算覆蓋，不一致只記診斷，不影響結論。

25 案例 / 83 可復現缺陷，實跑 83/83 檢出、0 誤報、0 不可判定 (§5.3)。

證據分層，并把"沒做的"寫出來。五層證據表把"時序分析 / 布局布線 / 上板驗證"顯式標"未運行"，避免把"仿真通過"或"綜合通過"誤讀為"能上板" (§5.5)。

離線可完整復現。自帶離線調試模型服務（只監聽回環、不需要密鑰、不聯網），評委可在無憑據環境下跑通全部流程與實驗。

驗收一份 RTL 不需要先寫測試台。iverilog-ai verify-diff 一條命令判斷兩份 RTL 行为是否一致（合約與計劃都可自動準備），產出可直接貼進 PR 的對比報告。

結論口徑分四層、措辭互不重疊。運行狀態 / 比對情況 / 設計結果 / 對比結論各用一套不共用的詞，避免同一個 passed 被讀成三件事 (§4.2)。

二、需求分析與總體方案

2.1 需求分析

(1) 現有做法與痛點

现有做法	主要痛点	需求优先级
手写 testbench	工作量大、边界易漏、换设计即重写	覆盖度
通用大模型直接写 testbench	期望值不可信；无法离线复核；结论不可追溯	可信度
商业仿真器的 AI 助手	闭源、需授权、无法审计判决路径	可信度
上游项目"没有测试台"就放弃回归	重构与 AI 改写无法验收	可信度

我们的优先级排序是 可信度 > 覆盖度 > 效率：一个"看起来自动化了但结论不可信"的工具，在验证场景里比没有工具更危险。

(2) 需求量化（每一行都对应一处可核验的实现）

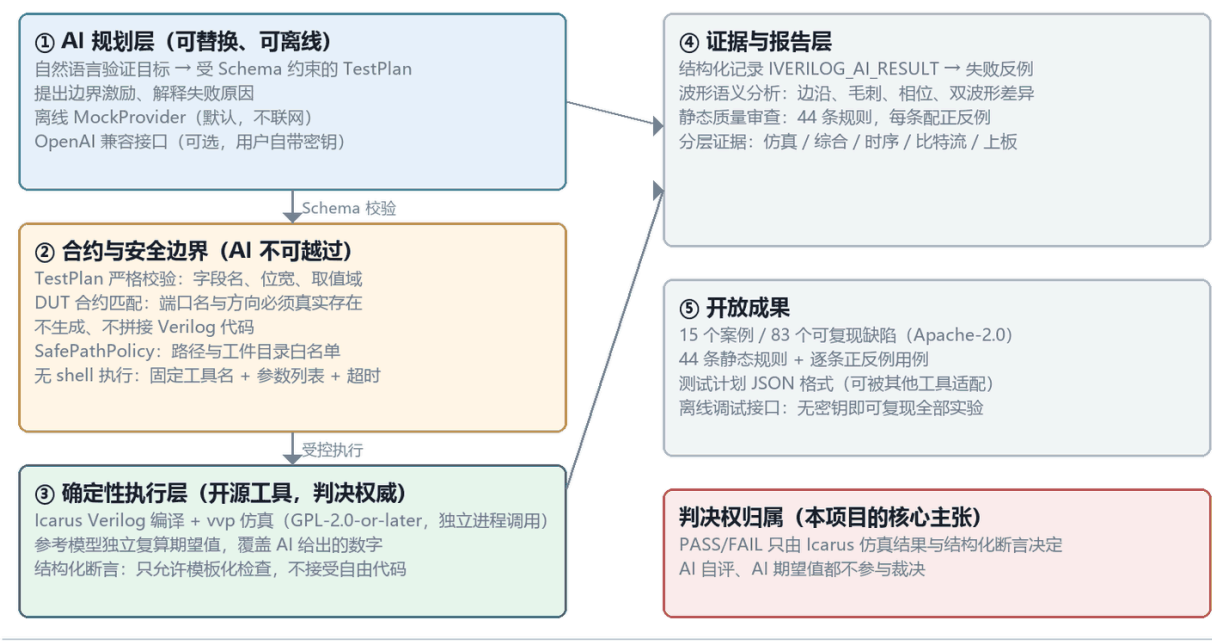
需求	目标	说明
结论必须可信	AI 期望值一律由独立参考模型复算覆盖	第三门；SUPPORTED == AUTHORITATIVE 有断言守着
结论必须可读	四层措辞互不重叠，不出现"同名不同义"	core/labels.py 为唯一措辞来源
不要求用户先写测试台	从 RTL 自动提合约、按合约生成测试计划	verify-diff 默认路径
判定必须可复现	同一输入 → 同一结论；退出码稳定	598 项测试；5 个矩阵脚本
判定必须可追问	每条失败落到"第几拍 / 期望 / 实际 / 失败指纹"	结构化记录 + 失败解释
成本学生可承担	全流程本地、离线可跑；在线开销透明	无密钥跑通完整 AI 路径；费用按核验价目表给区间
能接进别人的流程	命令行退出码 + GitHub Action	verify-diff 的 0/1/2；action.yml

(3) 未达标项（如实列出）

对象 ③④ 的需求未做正式调研，本文只用试用反馈与课程场景推断；
对象 ④ 的批量批改已实现（iverilog-ai grade），但没有在真实班级里试用过；
正文 19 页，超出大纲建议的 15 页（见文首说明），我们没有为此删除证据；
上游 Issue / Pull Request 尚未提交（见 §6.3）。

2.2 总体方案

图 1 系统架构：AI 能影响什么，不能影响什么



AI 只能产出行 ① 的结构化测试计划；②③ 两层是确定性代码，AI 无法改写执行内容与判决口径。

图 1 系统架构

系统分四层，关键设计是把 AI 关在第一层里。各层职责如下：

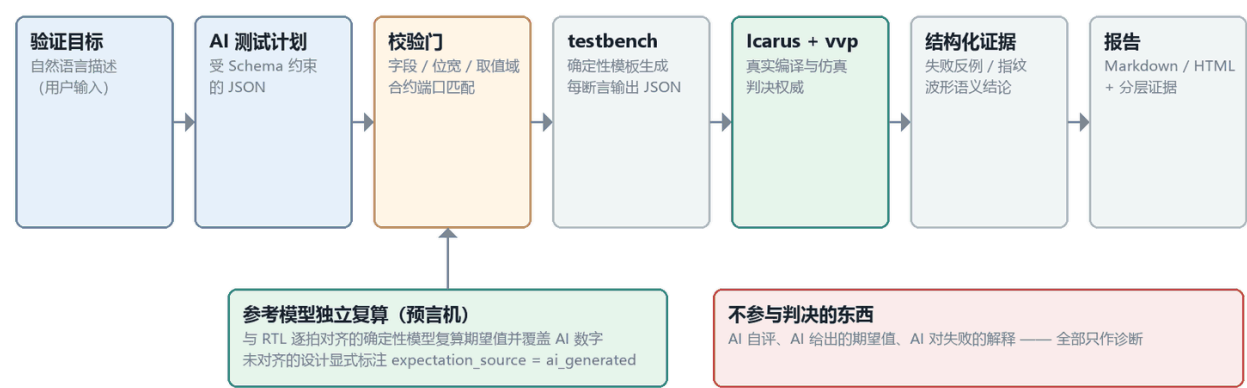
层	职责	AI 能影响吗
① AI 规划层	只输出结构化 JSON 测试计划；离线时用按合约生成激励的确定性规则引擎，需要真实模型能力时接任意 OpenAI 兼容接口	能（唯一——层）
② 合约与安全边界	字段/位宽/取值域严格校验 + 与 DUT 合约端口逐一对齐；不生成、不拼接 Verilog；路径白名单；执行不经 shell	不能
③ 确定性执行层	Icarus 编译 + vvp 仿真作为唯一判决权威；参考模型独立复算期望值并覆盖 AI 数字	不能
④ 证据与报告层	结构化记录、波形语义分析、静态质量审查、分层证据表、四层结论口径	不能

一次验证的数据流（10 步，每一步的产物都可单独审计）：

步骤	发生什么
1	用户给出验证目标（一句话）与 DUT 合约（端口/时钟/复位）
2	AI 规划层据此产出 TestPlan JSON：每条向量写明驱动哪些端口、跑几拍、期望什么

3	门 1：严格 Schema 校验（字段名、类型、位宽、取值域）——杜撰字段或越界取值直接拒绝
4	门 2：与 DUT 合约逐端口对齐——端口名或方向对不上直接拒绝
5	门 3：参考模型按同一份激励独立复算期望值，覆盖 AI 给出的数字
6	门 4：结构化断言只允许五种受限模板，模型无法注入 Verilog/SVA 代码
7	确定性模板生成 Verilog-2001 testbench；每条断言输出一行 IVERILOG_AI_RESULT {json}
8	Icarus 编译 → vvp 仿真（固定参数列表、不经 shell、带超时与输出上限）
9	严格解析器消费结构化记录；结构化断言在采样序列上判定；波形语义分析读 VCD
10	产出报告（四层结论口径 + 证据等级）、失败反例（带失败指纹）、VCD 与证据包

图 2 验证流程：从自然语言验证目标到可核验报告



所有结论可回溯到 run 目录下的编译日志、仿真日志、VCD 与 result.json。

图 2 验证流程

2.3 可行性与成本

学生团队可承担——这是本项目的一条硬约束，每一项都有对应的实现：

维度	数据	结论
算力	不需要 GPU、不需要训练模型；全部实验在本机 CPU 完成	普通笔记本即可

软件	全部开源免费 (Icarus / Python / pydantic / Streamlit; Yosys 有 WASM 版, 离线可跑)	零授权成本
单例耗时	"生成计划 → 生成 testbench → 编译 → 仿真 → 出报告"秒级; 15 案例 + 83 缺陷全矩阵分钟级	迭代快
离线能力	无密钥、不联网即可跑通完整 AI 路径 (含调试接口)	无网络也能演示
在线开销	20 轮真实模型实验的费用区间 \\$.2.6–5.2 (按核验过的官方价目表; 与 API 余额实测下降 ¥23.66 吻合)	学生自付得起
数据	不需要外部数据集; 基准集由团队自行编写并逐项实跑验证	无数据授权风险

三、AI 技术与开源及第三方资源选择

3.1 AI 技术的作用

AI 在本项目中承担三件事，每一件都配了对应的验证手段——这是本项目与"让模型直接写 testbench"最根本的分工差别：

(1) 把自然语言验证目标转成结构化测试计划。

技术实现：模型的唯一输出是一份 JSON (vectors[].inputs 驱动哪些端口、跑几拍, expected 期望什么)，受严格 Schema 约束；提示词里内置全部 44 条规则的上下文与实测编码约定。

为什么需要 AI / 怎么验证：边界条件的枚举空间大且依赖设计语义，人工穷举成本高——这正是生成式模型擅长的"按语义提出候选假设"。验证方式是四重门控（下表），以及消融实验给出的量化答案：AI 计划 + 权威预言机可检出 76/83，而关掉预言机是 0/83。

(2) 提出边界激励。

技术实现：模型在计划里显式写出每条向量的 rationale（人读的意图说明），例如"FIFO 写满" "计数器回绕" "101 序列重叠检测"。

怎么验证：与仓库自带的人工边界激励做同口径对照。实测：通用激励比人工边界激励少检出 7 条，且少检出的全是"要精确时序或边界序列才能触发"的缺陷 (§5.3) ——这个结论对我们不利，但它是"AI 能不能替代人工测试设计"这个问题唯一的硬答案。

(3) 解释失败原因。

技术实现：把结构化失败记录（检查项 / 第几拍 / 期望 / 实际）翻成"下一步看哪里"，指出该信号在 RTL 里被赋值的行号，并给出复现命令。

怎么验证：措辞上严格守住一条线——只写"是「被赋值的位置」"，不一定是「出错的

位置」"（问题也可能在激励、复位或上游信号），并有测试钉住这句。

四重门控——AI 输出的验证链（这是本项目的核心工程内容）：

门	机制	拦截什么
第一门	严格 JSON Schema 校验（字段名、类型、位宽、取值域）	格式错误、杜撰字段、越界取值
第二门	DUT 合约匹配（端口名与方向必须真实存在）	模型杜撰端口别名（如把 enable 写成 en）
第三门	确定性参考模型复算并覆盖 AI 期望值	AI 写错期望值导致假通过或假失败
第四门	结构化断言只允许五种模板化检查	模型注入自由 Verilog/SVA 代码

第三门是决定性的。内置案例的参考模型都已与参考 RTL 逐拍对齐（15/15，多个随机种子零差异），因此可以直接用模型复算值作为权威期望值。它的作用不是理论上的——消融实验里把它关掉，检出率从 76/83 直接掉到 0/83。

3.2 开源及第三方资源选择

完整的名称、版本、来源、许可证、使用方式、关键义务与自研边界的逐项清单，见随报告提交的《开源及第三方资源使用清单》（附件）。选型依据摘要：

资源	选它的理由	许可证	使用方式
Icarus Verilog	唯一成熟、完全开源、可脚本化、支持 Verilog-2001/-2012 的仿真器	GPL-2.0-or-later	独立进程调用，不分发；判决权威
Yosys / YoWASP	开源综合工具；WASM 版本可完全离线运行	ISC	独立进程调用，不分发；可选证据层
pydantic	严格 Schema 校验，是把 AI 输出关进笼子的关键依赖	MIT	库依赖
Streamlit	零前端成本的单页演示，评委可在浏览器里直接复现	Apache-2.0	库依赖（可选）
GTKWave	可选的人工波形复核	GPL-2.0-or-later	独立进程调用，核心流程不依赖

3.3 自主开发与权利边界

类别	内容
第三方开源提供	仿真引擎（Icarus）、综合引擎（Yosys）、Schema 校验库（pydantic）、页面容器（Streamlit）。均为独立进程调用或库依赖，未修改、未分发其代码

通用 AI 工具完成	辅助编写部分 Python 模块、Verilog 测试台与文档草稿；生成内容不主张为团队自主知识产权，且其正确性一律由团队的编译、仿真与测试逐项验证
团队自主设计/开发/验证	测试计划 JSON 格式与校验规则；确定性 testbench 生成器；参考模型与逐拍对齐方法；结构化断言模板；基准集（15 案例 / 83 缺陷）与其边界 testbench；44 条静态规则及逐条正反例；波形语义分析（边沿/毛刺/相位/差异）；分层证据层；行为对比工作流 verify-diff + 可复用 GitHub Action；四层结论口径与措辞表；批量批改与相似度线索；安全边界（路径策略、无 shell 执行、超时与输出上限）；全部中文文档与实验记录

3.4 融合方式：为什么不是"简单调用接口"

AI 输出必须穿过确定性边界才能执行。模型输出必须通过 Schema 与合约双重校验，再由确定性代码生成 testbench；模型从不接触 Verilog 语法层，因此结构上不可能注入代码、命令或路径。

开源仿真器是判决权威，AI 只是假设提出者。即使模型完全不可靠，工具仍然可用（退化为"人工设计激励 + 真实仿真裁决"）。

参考模型独立复算，堵住"写错期望值"这条路。这是同类工具普遍缺失的一环，也是我们在真实模型实验中观测到问题之后专门补上的一环——其作用见 §5.3 的消融实验。

四、设计与实现过程

4.1 核心功能与流程

功能	输入 → 输出
测试计划生成与校验	自然语言目标 + DUT 合约 → 通过 Schema 校验的 TestPlan
确定性 testbench 生成	TestPlan + 合约 → Verilog-2001 testbench（每条断言输出一行结构化 JSON）
执行与取证	testbench + RTL → 编译/仿真日志、结构化记录、失败反例、VCD
权威预言机	TestPlan + 合约 → 参考模型复算的期望值（覆盖 AI 数字）
结构化断言	受限模板（信号相等/稳定/永不置高） → 通过/失败记录
波形语义分析	VCD → 边沿统计、毛刺判定、相位偏差、双波形差异、失败时间窗

静态质量审查	RTL 源码 → 44 条规则命中 + 质量评分 + Markdown 报告
分层证据	RTL 源码 + Yosys → 综合状态、单元统计、五层证据表
行为级对比	两份 RTL + 同一 TestPlan → 逐检查项差异 + 波形差异
行为对比 workflow	两个 RTL 文件 → 自动提合约草稿 + 生成计划 → 结论句 + 差异清单 + 可贴进 PR 的报告（不需要被测项目自带 testbench）
批量批改	一整个目录的提交 + 标准实现 → 逐份结论 + 两两相似度线索（区分“改名抄”与“直接抄”）
可复用 GitHub Action	PR 上 uses: 一行 → 行为回归结论写进 Job Summary / 产物 / PR 评论

4.2 关键实现

每个关键实现都按「技术实现 + 为什么这么做」两段写——后者才是评审真正想看的判断。

(1) 确定性 testbench 生成器。

技术实现：生成器不解析、不拼接模型提供的任何文本，只把已通过校验的标量数据填入固定模板；每条断言输出一行 IVERILOG_AI_RESULT {json}，由执行器的严格解析器消费。

为什么这么做：这样"AI 生成的内容"与"可执行代码"在结构上被彻底分离。不是靠提示词叮嘱模型"不要写危险代码"，而是让模型根本没有写代码的通道。

(2) 权威预言机门控。

技术实现：只有与 RTL 逐拍对齐的参考模型才能覆盖期望值；未对齐的设计返回空字典，调用方回退为 AI 期望值并如实标注 expectation_source="ai_generated"。

为什么这么做：模型一旦与 RTL 差一拍，就会把"参考设计通过"改写成"参考设计失败"，比没有预言机更糟。放宽这个集合的方法只有一个——让模型逐拍对齐（"对齐一个、加入一个"）。消融实验显示这一层是承重墙：关掉它，检出率从 76/83 掉到 0/83。

(3) IEEE 1364 非阻塞赋值语义对齐。

技术实现：把同一组向量同时喂给参考模型与 RTL，逐拍比较可观测输出，要求零差异。

为什么这么做：对齐过程中共发现并修掉 8 个模型错误，其中三个同一根因——同一时刻对同一变量的非阻塞赋值，其右侧读到的是旧值。例如 UART 发送器写 bit_idx <= bit_idx + 1 之后同拍读 frame[bit_idx+1]，用的是旧的 bit_idx；模型若用新值取位，输出就整体错开一拍。SPI 的 sclk/mosi、去抖的 key_state 都栽在同一处。这类问题只有逐拍对照才能发现，肉眼看波形很容易误判成"差不多对"。

(4) 路径与进程安全。

技术实现：所有外部进程以固定工具名 + 参数列表启动，不经过 shell；输入与工件路径走白名单策略；每个进程带超时与输出上限；不自动覆盖原始 RTL。

为什么这么做：工具要跑用户上传的 RTL，"不执行来路不明的代码"不能只写在文档里——必须让执行路径上根本没有拼接字符串的机会。

(5) 噪声校准作为交付物的一部分。

技术实现：静态规则与波形毛刺判据各经历三轮校准，逐条核实命中是否为真。

为什么这么做：初版规则对仓库内 84 个 RTL 文件报出 317 条命中，含大量误报

(如把注释结尾 / 当成除法、把合法写法误判为锁存器)。“规则命中数”本身不是成果，“命中里有多少是真的”才是——所以校准过程也写进交付物 (§5.4)。

(6) 三个真实事故的复盘。三条都来自真实使用(含本人自测与外部试用反馈)，每条都能在仓库提交历史里找到对应修复与回归测试。写进报告是因为“踩了什么坑、怎么定位、怎么防止复发”比一串功能清单更能说明工程能力，也说明判据确实在被真实场景压测，而不是只在测试集上好看。完整复盘表(症状 / 根因 / 修复 / 回归)见附录 §9.6。

三条各自暴露一类问题：① 离线路径喂了写死的演示计划(选择分支时没跟着合约走)；② Streamlit 的 widget 与 session_state 生命周期冲突(先建控件再写同名键必然报错)；③ 提示词与校验器各写了一份字段表——这一条最值得记：我们一度把它归因为“模型写错字段”，复核后发现提示词自己就把该字段列进了模板，模型只是照着写，缺陷在本项目一侧。报告按修复后的事实陈述，不把自己的缺陷记成模型能力问题。

4.3 团队分工与过程记录

成员分工：成员 A 负责 AI 规划层与 Provider 适配、测试计划 Schema 与校验；成员 B 负责确定性执行层、安全边界与证据链；成员 C 负责基准集与缺陷设计、参考模型对齐与文档。三人共同完成网页演示与实验设计。评审材料按竞赛规则第(九)款做匿名处理。

过程记录：项目以版本迭代方式推进，仓库提交历史即完整过程记录(八个阶段，每阶段有对应的提交信息与验证结论)，阶段时间线见附录 §9.7。

生成式 AI 与智能体工具的使用：用于辅助编写部分 Python 模块、Verilog 测试台与文档草稿。团队的审核、修改与验证工作包括：逐文件人工审核；以本机 Icarus 真实编译与仿真结果作为功能结论的唯一依据；以 598 项自动化测试作为回归门禁；对每个缺陷变体单独复现失败证据；核对每条技术陈述与实验产物的一致性，删除与实现不符的表述。对所用模型/平台的许可与服务条款：项目默认不请求任何网络服务，仅在用户显式配置密钥并打开网络开关后才调用，且不在仓库内保存凭据。

AI 生成内容的权利核查：仓库内无第三方复制代码；未使用第三方图片、音视频或数据集；报告插图由团队用自有脚本(scripts/make_report_figures.py)程序化生成。

五、成果展示与效果验证

5.1 成果展示

作品提供命令行与网页两个入口。网页包含：AI 接口设置、测试计划查看、执行流水线、失败反例、波形语义结论、分层证据、静态质量审查、自定义 RTL 导入与行为级对比。

页面顶部固定展示"实证状态"面板（案例数、缺陷数、规则数、测试数、已对齐模型数），全部数字由页面实时读取仓库文件现算。

复现方式见 §9 附录；网页入口：streamlit run ui/app.py。

5.2 验证方法

方法	做法	能回答的问题
固定矩阵基准	15 个参考设计 + 83 个缺陷变体，逐个跑同一固定激励	能不能检出已知缺陷，会不会误报
消融实验	同一批 83 个缺陷，三档配置只改一个变量（激励来源 / 期望值来源）	AI 那一环到底值不值、哪一环在承重
逐拍对齐	同一组向量同时喂给参考模型与 RTL，多随机种子逐拍比较	权威期望值本身是否正确
真实模型公平实验	固定 / 随机 / 离线AI / 在线AI 四种策略跑同一批案例	AI 到底带来多少提升，代价是什么
负例验证	构造"必须失败"的输入（不可综合 RTL、语义等价重写、已知缺陷、编译不过的候选）	判据是否真的在起作用，而不是永远说"通过"
自动化回归	598 项测试（含实测 Icarus 的端到端用例）+ mypy / 死代码 / 编码损坏门禁	改动是否破坏既有结论
外部复核	10 分钟复核指南（docs/reproduce_in_10_minutes.md）：5 条命令 + 期望数字 + 对不上怎么反馈	别人能不能自己验一遍

测试条件（每个数字都在这个条件下取得，换条件就要重测；完整表见附录 §9.10）：Icarus Verilog 12.0（-g2012）、Windows 11 x64 + Python 3.11（CI 另跑 Linux）、单例超时 30 秒、无随机性（离线规划器固定种子，同一命令同一输入必得同一结论）、全程不需要密钥也不需要联网。超时与编译失败一律记为"不可判定"，不折算成检出。

5.3 验证结果

图 4 基准矩阵实跑结果（固定向量，可复现）



图 4 基准矩阵结果

(1) 固定矩阵基准（可一键复现）

指标	结果
参考设计通过	15 / 15
缺陷检出	83 / 83（检出率 100%）
参考误报	0
不可判定	0

(2) 消融实验：AI 规划到底有没有用，哪一环在承重（python scripts/run_ablation_matrix.py）

上面那张表只能说明"手写 testbench 能检出全部缺陷"，回答不了"AI 那一环值不值"。
我们把两个因素拆开测：同一批 83 个缺陷，三档配置，只改一个变量。

配置	激励来源	期望值来源	缺陷检出	参考误报	不可判定
A handwritten-tb	人工设计的边界激励（仓库自带 testbench）	人工写死在 testbench 里	83 / 83	0 / 15	0
B ai-plan-plain	离线规划器按合约生成	没有期望值	0 / 83	0 / 15	0
C ai-plan-oracle	离线规划器按合约生成	参考模型独立复算并覆盖	76 / 83	0 / 15	0
A ∪ C（推导值）	两者并用	权威期望值	83 / 83	0 / 15	—

读法（三行差异就是结论）：

B. → C: 0 → 76。权威预言机是承重墙。两行的激励完全相同（同一份计划、同一个随机种子），只把期望值从"没有"换成"参考模型独立复算"。结果是：没有预言机时，AI 生成的计划连一条可比的检查项都产不出来——83 条缺陷一条都没检出。这直接量化了第三门的作用。

A → C: 83 → 76。AI 的通用激励还顶不上人工边界激励。两行的期望值都是权威的，差别只在激励。逐条看少检出的这 7 条，全部是"要精确时序或边界序列才能触发"的缺陷：异步复位在两沿之间断言（mod10_counter）、同步器级数与释放窗口（sync_reset ×2）、抖动序列长度（debounce）、展宽期内重触发（pulse_stretcher）、写满 off-by-one（sync_fifo）、紧急抢占时刻（traffic_light_emergency）。逐条明细见附录 §9.8。
这就是本项目不声称"AI 可以替代人工测试设计"的量化理由：AI 能把写 testbench 的成本降下来，但边界激励的设计判断力仍要人来出。

A ∪ C = 83/83，规划器独立贡献 0 条。这个结论对我们不利，但必须如实写：

当前离线确定性规划器的价值不在"发现人没想到的缺陷"，而在"不用人手写 testbench 就能拿到 76/83"。（真实模型的激励质量高于离线规划器——见下面第（4）项在线实验——但同样需要预言机兜底，理由就是第 1 条。）

(3) 参考模型对齐：15/15 个内置案例的参考模型与 RTL 在 4 个确定性随机种子上逐拍零差异，因此全部可以作为权威预言机；建模覆盖与权威集合完全一致（有测试断言防止放宽）。

(4) 真实在线模型实验（当前口径：13 案例 / 67 缺陷 × 每案例 10 次重复 × 2 个模型，每个模型

130 次真实请求；详见 docs/experiment/model_comparison_2026-09-12.md。
下表给出单轮平均检出率——与只跑 1 轮的基线可比；括号内为 10 轮累计并集）：

策略	单轮检出率	累计并集	说明
固定向量（1 轮）	83.6%	56/67	人工设计的固定激励
随机向量（1 轮）	76.1%	51/67	同种子可复现的随机激励
离线 AI 规划（1 轮）	76.1%	51/67	确定性离线模型，用于验证流程而非能力
在线 AI（flash，10 轮）	85.8%	65/67（97.0%）	比固定向量高 2.2、比离线基线高 9.7 个百分点
在线 AI（v4-pro，10 轮）	87.8%	64/67（95.5%）	单轮更稳，但累计并集略低；生成慢 3.4 倍

该轮参考误报为 0（125 次参考设计运行无一次硬失败）。同时公开的未达预期项：2 个缺陷未被检出，原因已定位且都不是模型能力问题，而是激励相位/观测点——这两条在 手写边界 testbench 的固定矩阵里都能检出；另有 5 次请求（3.8%）的计划被严格校验拒绝，按“不可判定”如实记录，不折算成检出。其中一次拒绝的根因经复核在本项目一侧（提示词与校验器各写了一份字段表），已修复并加回归门禁，详见附录 §9.6 事故 3。
费用：\ \$2.6–5.2（两模型合计，按核验过的官方价目表换算，scripts/estimate_model_cost.py），与同期 API 余额实测下降 ¥23.66 吻合；两模型的漏检集合几乎不重叠（并集 66/67），多模型的收益来自互补而非选最强。

（5）负例验证（证明判据不是“永远通过”）：

不可综合的 RTL（变量上界的 while 循环）→ 综合层报 失败；
语义等价但写法不同的重写（多出一个内部辅助信号）→ 行为对比判 一致；
已知缺陷（极性反转、计数差一）→ 行为对比判 不同，并定位到具体信号与拍；
候选编译不过 → 行为对比判 未取得可比证据（退出码 2），而不是“行为不同”——
这两件事必须分开：前者是“没跑起来”，后者是“跑起来了但结果不同”。

（6）尚达到不了的目标：不做时序签核、不做代码覆盖率、不做 CDC 形式化、参考模型只覆盖内置案例——完整边界清单见 §7.4。

（7）量化成果指标（收尾总表：每一行都能在仓库里复现）

维度	数据	成果
正确性	15 案例 / 83 缺陷，83/83 检出、0 误报、0 不可判定	具备可核验的缺陷检出能力，且不靠猜
可信度	关掉参考模型预言机，AI 计划检出 76 → 0	量化证明“AI 的数字不作数”是架构而非口号
可复现	598 项自动化测试 0 失败；5 个矩阵脚本一键重跑	同一输入同一结论，无随机性

可解释	每条失败落到"第几拍 / 期望 / 实际 / 失败指纹"; 失败被翻成"看哪一行"	结论可追问, 学生/评审都能自己看
成本	离线路径 0 元; 20 轮真实模型实验 \ \$2.6-5.2 (与余额实测 ¥23.66 吻合)	学生团队可承担
开放	Apache-2.0; CLI + 库 + 网页 + 可复用 GitHub Action; 基准集与规则包	他人可直接复用, 含 PR 级行为回归
边界	不做时序/形式化/覆盖率; 参考模型只覆盖内置案例; 范围之外明确不说	不夸大——把没做的写在同一张表里

5.4 噪声校准记录（为什么这部分也算成果）

静态规则与波形毛刺判据都经历了三轮校准才做到不误报：规则从"9 条规则报出 317 条命中、含 error 级误报"收敛到"44 条规则、163 条命中、0 条 error 级误报"；毛刺判据从"窗口内翻转多次即不稳定"（在时钟上全面误报）改为"全局基线 + 连续 ≥3 个异常间隔 + 复位沿不参与统计"。逐条根因见附录 §9.11。

我们把这两段过程写进报告，因为"规则命中数"本身不是成果，"命中里有多少是真的"才是。

5.5 分层证据

图 3 分层证据：做过的与没做的都写出来

功能仿真	本次流水线提供	Icarus 编译 + vvp 执行 + 结构化断言
逻辑综合	已提供（可选）	Yosys 通用门级映射 + 单元统计（实测 98/98 个 RTL 变体通过）
时序分析	未运行	需要目标器件时序库与时钟约束 —— 本项目不提供
布局布线 / 比特流	未运行	需要厂商工具链（Vivado / Quartus 等） —— 本项目不提供
上板验证	未运行	需要实际硬件与测试装置 —— 本项目不提供

综合单元统计示例：PWM 5 个通用门级单元；UART 29 个；FIFO 34 个。

综合通过 ≠ 时序收敛 ≠ 能上板；本项目不做时序签核，也不给出频率结论。

图 3 分层证据

按需追加 Yosys 综合：98 个 RTL 变体（15 参考 + 83 缺陷）全部可综合，并给出通用门级单元统计（PWM 5 个、UART 29 个、FIFO 34 个）；复现见 §9.9。这张表刻意把"没做的"写出来：时序分析、布局布线/比特流、上板验证三行标"未运行"，读者就不会把"综合通过"误读为"能上板"。

一个值得记录的实测结论：#5 q <= d; 这类延时不会让 Yosys 报错——它被静默忽略。所以"综合通过"也不能反过来当作"这段代码的时序语义被正确实现了"的证据。

六、创新性与开放价值

6.1 创新性

与"让大模型直接写 testbench"的常见方案相比，本项目的实质改进有三条：

(0) 与已有方案的对比选型

方案	谁在判对错	不需要用户写 TB	判定可复现	成本	我们相对它的差异
人工手写 testbench	人 + 仿真器	✗	是	每个设计 80 行起	我们用同一套判据免掉写 TB，但保留人工边界激励的位置（消融实验显示它仍更强）
通用大模型直接写 TB	模型自己（它同时给期望值）	✓	否	低	判决权从模型手里拿走：AI 写的期望值被参考模型复算覆盖，不一致只记诊断
商业仿真器的 AI 助手	仿真器	部分	是	授权费 + 闭源	我们全链路可审计（判决路径、四层结论口径、逐条证据），且离线可跑、零授权成本
形式化等价验证	求解器	✓	是	需形式化建模	我们不做等价证明，只证明"在测试计划覆盖范围内行为一致"——门槛与成本低一个量级
本项目	开源仿真器 Icarus + 确定性参考模型	✓	是（无随机性）	零（全程本地离线）	唯一同时做到"AI 参与、判决权不在 AI、范围之外明确不说"的一档

把"AI 自评"从判决链里彻底移除。判决权归开源仿真器，AI 只提出假设；AI 给出的期望值会被确定性参考模型复算覆盖。这不是提示词技巧，而是架构上的分工。

证据分层与负向声明。主动写出未做的证据层级（时序/比特流/上板），并把"综合通过 ≠ 时序收敛 ≠ 能上板"作为固定免责声明写入报告与网页。

噪声校准与失败公开成为交付物的一部分。静态规则与波形分析都公开了误报校准过程；在线模型实验公开了 9 个未检出缺陷及逐条原因，而不是只报最佳成绩。

6.2 开放成果

成果	内容	许可
代码仓库	完整工具链（AI 规划层、执行层、证据层、网页）+ 可复用 GitHub Action	Apache-2.0
基准集	15 个案例 / 83 个缺陷变体 + 边界 testbench + 清单	Apache-2.0
规则包	44 条静态规则（含逐条正反例）+ 按案例组织的模型约束规则	Apache-2.0
测试计划格式	结构化 JSON 合约，可被其他工具或仿真器适配	Apache-2.0
中文文档	架构、规则清单、波形判据、对齐方法、分层证据、行为对比、10 分钟复核指南	Apache-2.0

6.3 社区贡献与复用

如实说明：目前尚无向 Icarus Verilog 上游提交的 Issue 或 Pull Request。本项目是围绕上游的非官方扩展，与上游维护者无隶属或背书关系，也不修改、不分发上游代码。

可复用方式（不需要上游改动）：① 基准集覆盖计数器、状态机、FIFO、UART、SPI、握手、去抖、PWM、复位同步等常见模块的 83 个缺陷变体，可直接用于评测其他验证工具或教学；② 测试计划 JSON 格式与"AI 提出、工具裁决"的分工可迁移到其他开源仿真器（Verilator、GHDL 等），换掉执行层即可；③ GitHub Action 让上游项目在 PR 上直接获得行为回归结论（docs/upstream/verify-diff-action.md）；④ 离线调试接口让任何人在无密钥、不联网的环境下复现全部流程与实验。

6.4 维护计划

短期继续扩展案例与缺陷类型（优先协议类与跨时钟域类）；中期把更多参考模型纳入逐拍对齐、为其他开源仿真器提供执行层适配；长期把基准集作为独立评测资源向上游社区介绍。问题反馈 走仓库 Issue 入口。

七、合规、安全与风险说明

7.1 开源及第三方许可合规

类别	关键义务	我们怎么做
GPL-2.0-or-later 工具（Icarus、GTKWave）	分发衍生作品时须提供对应源码	以独立进程调用：不修改、不复制、不重新分发、不链接，因此不触发源码提供义务；不声称上游背书；仓库不含其任何源码或二进制
ISC 工具（Yosys / YoWASP）	保留版权与许可声明	同上（独立进程调用），声明见 THIRD_PARTY.md

MIT / BSD / Apache-2.0 / PSF 依赖	保留版权与许可声明	逐项列在附件《开源及第三方资源使用清单》与 THIRD_PARTY.md
"公开可访问 ≠ 有再利用授权"	无明确许可不得复制/训练/再分发	未使用任何仅"可访问"但无明确许可的第三方数据集、图片或音视频；报告插图由自有脚本程序化生成

7.2 数据、权利与隐私

未使用任何第三方数据集；不收集用户数据；不涉及个人信息。用户上传的自定义 RTL 只在本机处理，不自动外发。发往模型服务的内容限于设计名、验证目标、DUT 合约摘要与脱敏后的失败字段；不上传 RTL 源码、文件路径、命令或密钥。用户自带密钥，仓库不打包、不绑定任何模型服务。

7.3 AI 输出、知识产权与内容安全

风险	应对措施
模型幻觉（杜撰端口、位宽、期望值）	四重门控：Schema 校验 → 合约匹配 → 参考模型复算覆盖 → 仅允许模板化断言
提示注入 / 代码注入	模型输出不进入命令、路径或代码；生成器不做字符串拼接；执行不经 shell
错误的"通过"结论	判决权归 Icarus；参考设计出现失败时单独计为误报指标，不计入检出
凭据泄露	默认离线、无密钥；密钥只存在于进程内存或页面会话；.gitignore 排除 .env；仓库检索无密钥
AI 生成内容的权利	不主张为团队自主知识产权；已说明参与环节与团队验证工作

7.4 技术限制与不宜使用的场景

仿真通过 不等于 可综合；综合通过 不等于 时序收敛或能上板；
不做 时序签核、不做 代码覆盖率、不做 CDC 形式化验证；
静态规则是保守的 lint 式检查（不做完整语法树解析，对宏展开与 generate 内动态结构判断能力有限，策略是"宁可漏报也不误报"）；波形分析只覆盖 VCD 中 dump 出来的信号；
参考模型只覆盖内置案例；自定义设计的期望值来源只能是 AI 或人工，工具会显式标注来源、可信度更低；自定义 RTL 不是安全沙箱，请勿用于执行来路不明的设计。

八、总结与展望

8.1 项目总结

本项目围绕 Icarus Verilog 构建了一套"AI 提出、开源工具裁决"的验证工具链：可运行的命令行与网页；15 案例 / 83 缺陷基准集（实跑 83/83 检出、0 误报、0 不可判定）；15/15 参考模型逐拍对齐；44 条静态规则（逐条正反例 + 噪声校准）；波形语义分析、分层证据、

行为对比三套深度能力；四档消融实验量化了"AI 那一环到底值多少"；598 项自动化测试作为回归门禁；全程离线可复现。

主要价值：把"AI 辅助验证"从"相信模型"变成"用开源工具裁决模型提出的假设"，并把这个过程中的失败、误报与边界如实公开——包括那条对我们不利的结论（规划器没有多检出任何一条人工边界激励覆盖不到的缺陷）。

未完成事项：时序与上板证据层（需器件库与硬件，超出本周期条件）；上游社区贡献；更广泛的第三方工具适配；对象③④的用户调研。

团队收获：理解了 IEEE 1364 非阻塞赋值的实际影响（8 个模型错误中有 3 个源于此）；学会把"工具报出的数字"与"数字里有多少是真的"分开评估；体会到"如实说明没做什么"比"多报一个指标"更能提高可信度。

8.2 后续计划

① 扩展案例与缺陷类型（优先协议类与跨时钟域类），并把参考模型对齐方法固化为可复用流程；② 为其他开源仿真器（Verilator、GHDL 等）提供执行层适配，使基准集与测试计划格式可跨工具复用；③ 完善教程与在线演示，降低他人复用门槛，并在权利边界清晰的前提下向上游社区介绍本项目与基准集。

正文完。以下为附录（不计入 15 页正文预算）， 按需提供。

九、附录

9.1 开源及第三方资源使用清单

见随报告提交的附件《开源及第三方资源使用清单》（仓库内：
docs/competition/opensource_resource_list.md）， 已按官方参考表列序编制。

9.2 仓库链接与运行说明

```
# 1. 安装（仅需 Python 3.11+ 与 Icarus Verilog）
python -m pip install -e .

# 2. 命令行快速验证（无需任何密钥、无需联网）
python -m iverilog_ai run `
--rtl rtl/mod10_counter.v `
--testbench tb/tb_mod10_counter.v `
--top tb_mod10_counter `
--iverilog D:\iverilog\bin\iverilog.exe `
--vvp D:\iverilog\bin\vvp.exe

# 3. 一键复现基准矩阵结论（15 参考 + 83 缺陷）
python scripts/run_benchmark_matrix.py

# 4. 启动网页演示
streamlit run ui/app.py

# 5. 无密钥、不联网地跑通完整 AI 流程
python -m iverilog_ai.ai.debug_server # 另开一个终端
```

推荐给评委的五分钟复现路径：跑第 3 步看基准矩阵结论 → 跑第 5 步 + 网页看 "AI 生成计划 → 受控生成 testbench → Icarus 裁决 → 报告"的完整链路 → 在网页上 切换到"自定义 RTL"体验导入与行为级对比。

9.3 测试记录与实验记录

记录	位置
基准矩阵逐项结果	仓库 .iverilog-ai/benchmark-matrix/
固定/随机/AI 策略公平实验	docs/experiment/
在线真实模型实验（含未检出缺陷逐条原因）	docs/experiment/online_model_experiment_2026-09-10.md
参考模型逐拍对齐方法与踩坑记录	docs/reference_model_alignment.md
静态规则清单与噪声校准	docs/static_rules.md
波形分析判据与三轮校准	docs/vcd_analysis.md
分层证据口径与实测	docs/layered_evidence.md
行为级对比判据与实测	docs/behavior_compare.md

9.4 插图说明

本报告插图（图 1–图 4）由团队自有脚本 `scripts/make_report_figures.py` 使用 Pillow 程序化生成，不包含任何第三方图片素材；脚本可重跑，图与仓库数据保持一致。

9.5 参考文献

IEEE Std 1364-2005, IEEE Standard for Verilog Hardware Description Language.

Icarus Verilog 项目文档与用户指南，<<https://steveicarus.github.io/iverilog/>>。

Yosys Open SYNthesis Suite 文档，<<https://yosyshq.readthedocs.io/>>。

lowRISC Verilog Coding Style Guide（仅参考公开文档中的编码与复位约定，未复制代码）。

cocotb 文档（验证工程实践参考，未复制代码）。

9.6 三个真实事故的完整复盘（正文 §4.2（6）的展开）

#	症状（用户原话 / 现象）	根因（代码位置）	修复	回归测试
1	选「离线」模式、案例选简单 ALU，点"执行 AI 计划并生成 testbench"报错 <code>vectors[0].inputs contains unknown port 'rst_n'</code> ；同一个案例点"执行真实 Icarus 仿真"却通过，看起来像离线模式坏了	离线分支直接把 <code>MockProvider()</code> 的默认返回值当计划，而那份写死的演示计划固定驱动 <code>rst_n</code> ——组合逻辑案例的合约里既没有时钟也没有复位	新增 <code>offline_provider(contract)</code> ：按当前合约生成激励（与本地调试模型服务同一个引擎），组合与时序案例都成立	<code>tests/core/test_offline_ui_plan.py</code> ：对每个内置案例都要求离线路径能产出可生成的计划
2	配置好 API 后点"从表格生成 JSON"→刷新表格→点"校验 contract"，报错 <code>st.session_state.custom_contract_json_text cannot be modified after the widget with key custom_contract_json_text is instantiated</code>	Streamlit 的 widget 一旦实例化就不允许再写同名 <code>session_state</code> 键；而"把规范格式写回文本框"必然要写它	把这两个动作移到 <code>on_click</code> 回调（回调在 <code>fragment</code> 主体之前执行），并去掉所有 <code>st.rerun(scope="fragment")</code>	<code>tests/core/test_ui_smoke.py</code> 加了 AST 门禁：禁止在这两处出现"先建控件再写同名键"的模式
3	生成测试计划报错 <code>assertions[0] contains unsupported field(s): signal</code> ；再点一次又能生成	提示词里给 <code>signal_implies</code> 列了 <code>signal</code> 字段，而校验器不接受——提示词与校验器各写了一份字段表；重试时换条路径恰好避开	字段表收成单一事实来源 <code>ASSERTION_FIELDS</code> （校验器、执行器、提示词共用）；重试时把拒绝原因追加进提示词	<code>tests/core/test_labels.py</code> 的"单一来源"门禁 + 断言模板门禁

9.7 阶段时间线（正文 §4.3 的展开）

阶段	主要工作	留下的可核验物
一	执行器与安全边界；固定矩阵基准成型	路径策略、进程超时、基准矩阵脚本
二	AI 计划 Schema 与确定性 testbench 生成	四门校验链、结构化记录格式
三	参考模型与逐拍对齐（发现 8 个非阻塞赋值语义错误）	15/15 对齐测试
四	真实模型公平实验（暴露"AI 期望值不可信"）	在线实验记录与口径更正
五	引入权威预言机覆盖，参考误报归零	参考模型复算 + 三态证据等级
六	静态规则与波形判据的三轮校准	44 条规则、逐条正反例
七	面向两类主打对象适配：verify-diff、GitHub Action、批量批改	消融实验、Action、复用文档
八	诚实性加固：四层结论口径、提交体检、10 分钟复核指南	措辞门禁、占位符门禁、复核指南

9.8 消融实验中规划器漏检的 7 条（正文 §5.3（2）的展开）

案例	缺陷	触发条件	通用激励为什么碰不到
mod10_counter	mod10_bug_async	复位在两沿之间断言	需要复位释放与时钟沿的精确相位关系；整拍边界给复位看不出异步与同步之差
sync_reset	srst_bug_single_stage	ext_rst_n 释放	差别只在释放路径的同步级数上，需要释放时刻恰好落在时钟沿附近
sync_reset	srst_bug_sync_assert_only	ext_rst_n 在两沿之间拉低	断言路径的同步与否只在释放窗口里可观测
debounce	deb_bug_counter_not_cleared	输入短暂回到采样值后再抖动	需要足够长的连续抖动序列才能让计数器累计出错
pulse_stretcher	pstr_bug_no_retrigger	展宽期内再次给脉冲	需要在展宽期内精确重触发，通用激励没构造重叠时序
sync_fifo	fifo_bug_full_off_by_one	写满 DEPTH 个数据	写入长度没卡在满/空边界上
traffic_light_emergency	traffic_bug_emergency_output	emergency=1	抢占时刻没落在那个状态拍上

9.9 可复现命令清单

想验证什么	命令
参考设计全过、缺陷全检出、零误报	python scripts/run_benchmark_matrix.py
AI 规划的两因素消融	python scripts/run_ablation_matrix.py

98 个变体全部可综合	python scripts/run_synthesis_matrix.py
离线 AI 路径 15/15	python scripts/run_pipeline_matrix.py
实验费用区间（按核验过的官方价目表）	python scripts/estimate_model_cost.py <实验目录>
全部自动化测试与类型门禁	python -m pytest -q / python -m mypy
两份 RTL 行为对比（不需要合约与计划）	python -m iverilog_ai verify-diff --baseline a.v --candidate b.v
批量批改一个目录	python -m iverilog_ai grade --dir submissions/ --golden rtl/ref.v
提交材料体检（页数 / 体积 / 元数据实名 / 占位符）	python scripts/check_submission.py report.pdf / --repo .

9.10 完整测试条件（正文 §5.2 的展开）

项	条件
仿真器	Icarus Verilog 12.0 (iverilog + vvp)，本机进程调用，-g2012
综合器	Yosys（可选层，不参与 PASS/FAIL 裁决）
运行平台	Windows 11 x64 + Python 3.11；CI 另跑 Linux（Ubuntu）与 Python 3.11/3.12
判决口径	只有 Icarus 的编译/仿真结果与结构化断言参与裁决；AI 的数字只作诊断
期望值口径	内置案例由参考模型复算覆盖（reference_model）；自定义 RTL 回退并如实标注
随机性	无。离线规划器 seed=0、激励表固定；同一命令同一输入必得同一结论
单例超时	30 秒（综合层 180 秒）；超时记为"不可判定"，不折算成检出
复现入口	python -m pytest -q 与 5 个矩阵脚本（§9.9），全程不需要密钥、不需要联网

9.11 静态规则与波形判据的三轮校准（正文 §5.4 的展开）

静态规则：初版 9 条规则、对仓库 84 个 RTL 文件报出 317 条命中，逐条核实后修掉：division-operator 把注释结尾 / 当成除法（84/84 全命中）、inferred-latch 把合法写法误判为锁存器（15 条 error 级误报）、missing-timescale 因注释剥离而全命中、时钟/复位名只匹配前缀（rst）导致实际命名 rst_n 从不命中。校准后：44 条规则、163 条命中、0 条 error 级误报。

波形毛刺判据：初版"窗口内翻转多次即不稳定"在时钟与计数器上全面误报；第二版 "短于全局中位数 1/3"仍把 busy 信号的正常窄脉冲报成毛刺；第三版改为"全局基线 + 连续 ≥3 个异常间隔 + 复位沿不参与统计"后，时钟、计数器、脉冲、双峰节奏全部不再误报，而真实连续快翻仍能命中。